Ripasso shell

1. Gli spazi

La shell è un interprete di **comandi**. Ogni riga viene divisa in parole separate da spazi, la prima parola è il comando e le altre sono i suoi argomenti.

Nell'assegnamento: NESSUNO spazio

```
count=0        # corretto
count = 0      # SBAGLIATO: esegue il comando "count" con argomenti "=" e "0"
count= 0       # SBAGLIATO: esegue "0" con count impostata a vuoto
count =0       # SBAGLIATO: esegue "count" con argomento "=0"
```

L'errore che si ottiene è `count: command not found` oppure `count: not found`: la shell sta cercando un programma chiamato `count`.

Nelle condizioni: SPAZI obbligatori

`test` è un comando come `ls` o `grep`, e come ogni comando riceve argomenti separati da spazi. In una condizione `=` è uno di quegli argomenti: è l'operatore che confronta due stringhe. Quindi va scritto con lo spazio prima e dopo, al contrario dell'assegnamento.

```
if test "$a" = "$b"       # corretto: tre argomenti, "$a", "=", "$b"
if test "$a"="$b"         # SBAGLIATO: un solo argomento, la stringa
"ciao=mondo"
```

La seconda forma **non dà errore**: `test` con un solo argomento risponde vero se la stringa non è vuota, e `"$a"="$b"` contiene almeno il carattere `=`, quindi la condizione è **sempre vera**. Lo script gira, non segnala niente ed entra sempre nel ramo `then`.

Dove lo spazio non basta: `then`

`then` non è un argomento di `test`: è un pezzo del costrutto `if`. Per separarlo dal comando che lo precede lo spazio non basta, serve un fine riga o un punto e virgola.

```
if test -f "$f"
then          # then su una riga sua...
if test -f "$f"; then     # ...oppure dopo un punto e virgola

if test -f "$f" then    # SBAGLIATO: "then" diventa il terzo argomento di test
```

L'errore è `syntax error near unexpected token 'fi'`, e indica la riga del `fi`, non quella sbagliata: la shell aspetta ancora il `then` e si accorge che manca solo alla fine del blocco.

---

2. Gli operatori di `test`

Confronti fra numeri

Operatore	Significato	Esempio
`-eq`	uguale (**eq**ual)	`test $# -eq 2`
`-ne`	diverso (**n**ot **e**qual)	`test $# -ne 2`
`-lt`	minore (**l**ess **t**han)	`test $n -lt 1`
`-le`	minore o uguale (**l**ess or **e**qual)	`test $n -le 0`
`-gt`	maggiore (**g**reater **t**han)	`test $val -gt $max`
`-ge`	maggiore o uguale (**g**reater or **e**qual)	`test $val -ge $soglia`

Si usano **solo** con i numeri. Con le stringhe si usano `=` e `!=`:

```
test "$a" = "$b"     # stringhe uguali
test "$a" != "$b"    # stringhe diverse
test $a -eq $b       # SBAGLIATO su stringhe: "integer expression expected"
```

Test su file e directory

Operatore	Vero se
`-e`	esiste (file o directory)
`-f`	esiste ed è un **f**ile regolare
`-d`	esiste ed è una **d**irectory
`-r`	si ha il permesso di lettura
`-w`	si ha il permesso di scrittura
`-x`	su un file: eseguibile; **su una directory: attraversabile**

Sulle directory il permesso che serve per entrare è `-x`, non `-r`:

- `-r` consente di elencarne il contenuto
- `-x` di attraversarla con `cd`.

Operatori logici

Forma	Significato
`test A -a B`	**a**nd
`test A -o B`	**o**r
`test ! A`	negazione

---

3. I comandi principali

Di seguito i comandi che ricorrono più spesso negli esercizi.

```
# quante righe ha il file
count=`wc -l < "$f"`

# in quante righe compare una stringa
occ=`grep -c "$str" "$f"`

# alcuni campi delle righe che corrispondono
grep "$str" "$f" | cut -f2,4 -d "," >> "$LOG"

# il valore di un campo, se la riga è una sola
v=`grep "$str" "$f" | cut -f2 -d ","`

# la riga con il massimo di un campo numerico
max=`sort -n "$f" | tail -n 1`

# il nome del file senza estensione
mese=`echo "$f" | cut -f1 -d "."`
```

Le opzioni che ricorrono

Comando	Opzione	Effetto
`grep`	`-c`	stampa il **numero** di righe che corrispondono, non le righe
`cut`	`-f`	quali campi: `-f1`, `-f2,4`, `-f2-5`
	`-d`	il delimitatore: **va sempre indicato**, il default è il tabulatore
`sort`	`-n`	ordine **n**umerico (altrimenti "10" viene prima di "9")
	`-r`	ordine inverso
`head` / `tail`	`-n N`	le prime / ultime N righe
`wc`	`-l`	conta le righe

Due dettagli:

- `wc -l < "$f"` è preferibile a `wc -l "$f"`: la seconda forma stampa **anche il nome del file**, quindi il risultato non è un numero e i confronti con `test` falliscono.
- `sort -n` ordina sul primo campo. Se il numero da confrontare non è il primo campo serve `sort -t "," -k2 -n`.

Sostituzione di comando e apici

```
count=`wc -l < "$f"`      # apici inversi: esegue il comando e ne prende l'output
count="wc -l < $f"        # SBAGLIATO: apici doppi -> count contiene la STRINGA, non il risultato
```

```
count='wc -l'              # SBAGLIATO: apici singoli -> nessuna sostituzione,
nemmeno delle variabili
```

Gli apici doppi attorno alle variabili (`"$f"`) servono quando il valore può contenere spazi: senza, un file chiamato `mio file.txt` diventa due argomenti.

E gli apici vanno attorno a **ciascun** argomento:

```
script_ric.sh "$dir" "$categoria"     # corretto: due argomenti
script_ric.sh "$dir $categoria"       # SBAGLIATO: un solo argomento
```

Aritmetica

Non esistono `count++` né `count += 1` :

```
count=`expr $count + 1`    # gli spazi attorno al + sono obbligatori
count=`expr $count+1`      # SBAGLIATO: expr riceve una sola parola
```

---

4. La struttura in due script

Gli esercizi d'esame hanno in genere la stessa struttura: **due script** con i compiti divisi.

Lo script principale si esegue una volta sola:

- controlla gli argomenti;
- prepara ciò che serve alla ricorsione e la avvia;
- alla fine stampa il valore richiesto.

Lo script ricorsivo si esegue una volta per directory visitata:

- entra nella directory ricevuta;
- fa il lavoro sui file che le appartengono;
- poi si richiama su ciascuna sottodirectory.

Non ricontrolla gli argomenti, perché quando viene invocato sono già stati validati.

```
# --- script principale ---
# 1. controlli sugli argomenti: numero, tipo, path relativo/assoluto,
esistenza, permessi, ecc...
arg1="$1"
shift
dir="$1"
shift

# 2. stato condiviso, tutto con export
PATH=`pwd`:$PATH
export PATH     # per poter invocare lo script ricorsivo

LOG="$HOME/..."
```

```
> $LOG          # sovrascrive o accoda: dipende dalla traccia
export LOG      # il file di log viene creato ed esportato qui una sola volta

MAX_COUNT=/tmp/...
echo '0' > $MAX_COUNT
export MAX_COUNT

MAX_NAME=/tmp/...
> $MAX_NAME
export MAX_NAME

# 3. prima chiamata
script_ric.sh "$arg1" "$dir"

# 4. stampa del massimo e rimozione dei file temporanei
```

```
# --- script ricorsivo: STESSI parametri, STESSO ordine ---
arg1="$1"
shift
dir="$1"
shift

cd "$dir"

for f in *.ext    # il lavoro vero, un file per volta
do
    if test -f "$f" -a -r "$f"
    then
        ...   # estrazione per il log, aggiornamento del massimo
    fi
done

for d in *     # la ricorsione
do
    if test -d "$d" -a -x "$d"
    then
        script_ric.sh "$arg1" "$d"
    fi
done
```

---

5. Gli errori ricorrenti

1. **Spazi attorno a `=`** nell'assegnamento, o spazi mancanti dentro `test` / `[ ]`. Vedi §1.
2. **Sintassi sbagliata**: `count++`, `exit(1)`, `if (…) { … }`
   Si scrivono `count=`expr $count + 1``, `exit 1`, `if … then … fi`.
3. **`$` nell'assegnamento**: si scrive `count=0`, non `$count=0`. Il `$` serve a **leggere** una variabile, non ad assegnarla.

4. **Permesso di esecuzione**: senza `chmod +x *.sh` si ottiene `Permission denied`.
5. **`PATH` dimenticato**, o impostato senza `export`: lo script ricorsivo non viene trovato, o funziona solo al primo livello.
6. **Variabili non esportate**: il figlio non vede il nome del file temporaneo e scrive altrove, o niente.
7. **`wc -l "$f"` invece di `wc -l < "$f"`**: il risultato contiene il nome del file e i confronti numerici falliscono.
8. **`>` dove serve `>>`**: `>` crea il file da zero, `>>` aggiunge alla fine. Sul log, nello script ricorsivo, serve sempre `>>`: con `>` ogni directory cancella quello che ha scritto la precedente e resta solo l'ultima.

---

6. Testare lo script

```
sh -n script.sh              # solo controllo di sintassi, non esegue nulla
sh -x script.sh arg1 arg2    # esegue stampando ogni comando: mostra dove si
ferma
```

`sh -x` è il più utile: stampa ogni comando con le variabili già sostituite, quindi si vede esattamente con quali valori è stato chiamato `test` e dove la variabile era vuota.

I file di test

La traccia descrive il formato dei dati ma non li fornisce: l'albero su cui provare lo script va creato a mano.

```
dir/
├── file1.txt
├── file2.txt
├── donotuse.log      -> estensione sbagliata: deve essere ignorato
└── subdir/
    └── subsubdir/
        └── file3.txt -> qui sta il risultato atteso
```

Mettere il massimo nel livello più profondo è utile: se la ricorsione non scende, la risposta cambia e l'errore si vede subito.